

Dijkstra's algorithm (lowest cost walk) ✓

- relies on Bellman's optimality principle (optimal path from s to t means optimal sub-paths)
- like BFS (normal queue, FIFO), but Dijkstra uses priority queue (picks the node with the smallest current distance)
- Dijkstra = BFS but smarter about which node comes next $\rightarrow$ faster for single-source sparse graphs?

Input: G = directed graph with costs
 s, t = two vertices

Output: $dist$ = map (container storing key-value pairs) associating to each vertex the cost of the min cost walk from s to it.

$prev$ = map associating each vertex to its predecessor (how we reached each node)

ex: $prev[t] = s$ (previous of t is s).

(node) (distance to node z)
 $\downarrow$ $\downarrow$
key value

maps in C++: `#include <unordered_map>`
`std::unordered_map<int, int> dist;` $\rightarrow dist[3] = 10$

Algorithm: PriorityQueue q ;

Dictionary $prev$

Dictionary $dist$

$q.enqueue(s, 0)$; put s in the priority queue with priority 0 (distance = 0)

$dist[s] = 0$; distance from start to nodes is 0.

$found = false$; becomes true when you reach the target t .

while not $q.isEmpty()$ and not $found$ do

$x = q.dequeue()$; x is always on top (bc it's a priority queue)

for y in $Next(x)$ do

if y not in $dist.keys()$ or $dist[x] + cost(x, y) < dist[y]$ then

$dist[y] = dist[x] + cost(x, y)$

$q.enqueue(y, dist[y])$

$prev[y] = x$

end if

end for

if $x == t$ then

$found = true$

end if

end while

! non-negative costs $\rightarrow$ Dijkstra works fine. (nodes processed once)

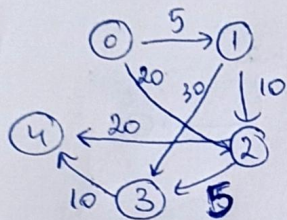
! negative costs, no cycles $\rightarrow$ Dijkstra works fine bc distances can improve later (nodes processed > times)

! negative cycle $\rightarrow$ Dijkstra fails (distances keep decreasing ∞)

Backwards Dijkstra.

$t \rightarrow s$

$s=0, t=4$



relaxation $\leftarrow$

	x	y	dist: dict	g: prior. queue	next: dict																				
initializ.			<table border="1"><tr><td></td><td></td><td></td><td></td><td>0</td></tr></table>					0	$\leftarrow (4,0) \leftarrow$	-															
				0																					
iteration 1	4			$\leftarrow \leftarrow$																					
iteration 1.1.		2	<table border="1"><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td></tr><tr><td></td><td></td><td>20</td><td></td><td>0</td></tr></table>	0	1	2	3	4			20		0	$\leftarrow (2,20) \leftarrow$	<table border="1"><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td></tr><tr><td></td><td></td><td>4</td><td></td><td></td></tr></table>	0	1	2	3	4			4		
0	1	2	3	4																					
		20		0																					
0	1	2	3	4																					
		4																							
iteration 1.2		3	<table border="1"><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td></tr><tr><td></td><td></td><td>10</td><td>10</td><td>0</td></tr></table>	0	1	2	3	4			10	10	0	$\leftarrow (3,10) (2,20) \leftarrow$	<table border="1"><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td></tr><tr><td></td><td></td><td>4</td><td>4</td><td></td></tr></table>	0	1	2	3	4			4	4	
0	1	2	3	4																					
		10	10	0																					
0	1	2	3	4																					
		4	4																						
it 2.	3			$\leftarrow (2,20) \leftarrow$																					
it 2.1.		2	<table border="1"><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td></tr><tr><td></td><td></td><td>15</td><td>10</td><td>0</td></tr></table>	0	1	2	3	4			15	10	0	$\leftarrow (2,15) \leftarrow$	<table border="1"><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td></tr><tr><td></td><td></td><td>3</td><td>4</td><td></td></tr></table>	0	1	2	3	4			3	4	
0	1	2	3	4																					
		15	10	0																					
0	1	2	3	4																					
		3	4																						
it 3.	2			$\leftarrow \leftarrow$																					
it 3.1.		0	<table border="1"><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td></tr><tr><td>35</td><td>25</td><td>15</td><td>10</td><td>0</td></tr></table>	0	1	2	3	4	35	25	15	10	0	$\leftarrow (0,35) \leftarrow$	<table border="1"><tr><td>2</td><td></td><td>3</td><td>4</td><td></td></tr></table>	2		3	4						
0	1	2	3	4																					
35	25	15	10	0																					
2		3	4																						
it 3.2.		1	<table border="1"><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td></tr><tr><td>35</td><td>25</td><td>15</td><td>10</td><td>0</td></tr></table>	0	1	2	3	4	35	25	15	10	0	$\leftarrow (1,25) (0,35) \leftarrow$	<table border="1"><tr><td>2</td><td>2</td><td>3</td><td>4</td><td></td></tr></table>	2	2	3	4						
0	1	2	3	4																					
35	25	15	10	0																					
2	2	3	4																						
it 4.	1			$\leftarrow (0,35) \leftarrow$																					
it 4.1.		0	<table border="1"><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td></tr><tr><td>30</td><td>25</td><td>15</td><td>10</td><td>0</td></tr></table>	0	1	2	3	4	30	25	15	10	0	$\leftarrow (0,30) \leftarrow$	<table border="1"><tr><td>1</td><td>2</td><td>3</td><td>4</td><td></td></tr></table>	1	2	3	4						
0	1	2	3	4																					
30	25	15	10	0																					
1	2	3	4																						
it 5	0			$\leftarrow \leftarrow$																					
x=0=5 $\Rightarrow$ stop.																									

$x=0=s \Rightarrow$ stop.

$\Rightarrow$ the min. cost walk from $s=0$ to $t=4$ has the cost = dict[0] = 30 and is built by using next dictionary:

$s=0, \text{next}[0]=1, \text{next}[1]=2, \text{next}[2]=3, \text{next}[3]=4=t$

$0 \xrightarrow{5} 1 \xrightarrow{10} 2 \xrightarrow{5} 3 \xrightarrow{10} 4$

0 1 2 3 4 5
↑
10 11 9 14 2 3

$V_i \rightarrow$ list string [data[0:m]]

DIVIDE and CONQUER

1. Write a recursive function that determines the nr. of even elem. found on odd pos.

Tarjan algorithm (SCC) x

- DFS $\rightarrow$ compute for each vertex the earliest ancestor vertex / lowlink
- SCC representatives: their lowlink = the representative.

Input: $G =$ directed graph

Output: comp = map associating to each vertex the ID of its SCC.

Subalgorithm DF(Graph G , vertex x , int & last_index, Set & visited, Map & index, Map & lowlink, Stack & st)

for y in $N_{out}(x)$ do $\hookrightarrow$ stack of current active vertices

if y not in visited then

visited.add(y); mark y as discovered

$++last_index$; increase discovery counter

$index[y] = last_index$; store discovery nr of y (when y was first seen)

$lowlink[y] = last_index$; y can at least reach itself

st.push(y); put y on stack

DF(y); explore recursively everything reachable from y .

end if

$lowlink[x] = \min(lowlink[x], lowlink[y])$; if child y can reach an earlier ancestor, then x can also reach it through y .

end for

if $lowlink[x] = index[x]$ then; SCC test

// x is the representative of a new SCC

// create new component, extracting vertices from st until including x

end if

Algorithm:

int last_index = 0; start discovery counter from 0

Set visited

for x in $G.vertices$ do

if x not in visited then

DF($G, x, last_index, visited, \dots$)

end if

end for

visited.add(y) ; ~~mark~~ as assigned.

g.enqueue(y) ; continue exploration from y

comp[y]=c ; current SCC

end if

end for

end while

end if

end while

• During DFS, each vertex is in one of 3 states:

1) not visited

2) in progress (on ^{recursion} stack)

3) processed (on processed stack)

• SCC representative: the first vertex that DFS touches from that SCC.

→ first visited vertex in that SCC

→ last finished (processed) vertex in that SCC.

• Reflexive-transitive closure of a graph = accessibility relation

Kosaraju algorithm (SCC)

• directed graph $\Rightarrow$ find groups of vertices s.t. every vertex in that group can reach every other vertex

• x, y are in the same SCC $\Leftrightarrow$ $\begin{cases} x \text{ reachable from } y \\ y \text{ reachable from } x \end{cases}$

• has two phases:

1) Phase 1: DFS forward $\Rightarrow$ stack (LIFO)

2) Phase 2: traverse reversed graph (new traversal)

Input:

G = directed graph

Output:

comp = map associating to each vertex the ID of its SCC.

Subalgorithm $DF1$ (Graph G , vertex x , Set δ visited, Stack δ processed)

for y in $N_{out}(x)$ do ; explore all outbound neighbours of $x \rightarrow$

if y not in visited then

visited.add(y) ; mark neighbour as visited

$DF1(y)$; continue DFS from it (y)

endif

end for

processed.push(x) ; push x on the stack only after finishing all descendants.

Algorithm:

Stack processed ; stores vertices ordered by finishing time

Set visited ;

for s in X do ; go through all vertices of the graph

if s not in visited then ; if s wasn't discovered in a previous DFS

visited.add(s) ; start a new DFS tree from s

$DF1(G, s, \text{visited}, \text{processed})$; all vertices reachable from s are explored

visited.clear() ; reset the visited set

Queue q ; used to collect one SCC at a time

int $c = 0$; each SCC has its own id.

while not processed.isEmpty() do

$s = \text{processed.pop}()$; process vertices in reverse finishing order

if s not in visited then ; start of a new SCC

$c = c + 1$; new component number

comp[s] = c ; associate to vertex s the id (c) of its SCC

$q.\text{enqueue}(s)$; put s in the queue so we can explore its SCC

visited.add(s) ; mark it as part of the component

while not $q.\text{isEmpty}()$ do ; as long as there are vertices in SCC.

$x = q.\text{dequeue}()$

for y in $N_{in}(x)$ do ; follow inbound edges $\rightarrow x$

if y not in visited then ; y not assigned to any SCC

BFS (graph traversal) ✓

Time: $O(n+m)$

each vertex in queue at most once

each edge checked at most once

- find all vertices accessible from s
- find a path from s to t
- find min-length path from s to t .

NR. OF EDGES (not costs)

Input: G = graph
 s = vertex

Output: accessible = set of vertices accessible from s .
prev = map associating each vertex to its predecessor. on a path from s to it.

Algorithm:

→ all vertices at dist = 1 ; → all vertices at dist = 2 ; → ...

Queue q ; queue because we process vertices in the order they were discovered

Dictionary prev ; if prev[y] = x → we reached y from x.

Dictionary dist ; dist[x] = min length of a walk from s to x .

Set visited ; already discovered vertices → never enqueue those again

$q.enqueue(s)$

visited.add(s) ; marked visited when discovered, not when popped!

dist[s] = 0

while not $q.isEmpty()$ do

$x = q.dequeue()$; take first vertex from the queue (FIFO)

for y in Nout(x) do ; outbound neighbours of x

if y not visited then

$q.enqueue(y)$

visited.add(y)

dist[y] = dist[x] + 1

prev[y] = x

end if

end for

end while

accessible = visited ; the visited set.



preserves increasing distance order
??

for (auto const& [y, edge] :
g.parse_out(x))

• first time BFS reaches a vertex is through a min-length path??!

Proof of correctness:

- 1) All returned vertices are accessible
- 2) Algorithm finishes
- 3) All accessible vertices are returned.

Min-length path : • vertices processed in increasing order of distance
• the first time a vertex is reached is through the shortest nr of edges.

... we know the best paths using $\leq k$ edges.

path = walk with no repeating vertices

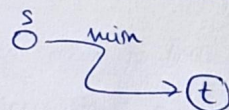
walk =

Bellman-Ford algorithm (lowest cost walk)

(space optimised version of the DP)

- no negative cost cycles

Input: G = directed graph with costs
 s, t = two vertices



Output: $dist$ = map associating to each vertex the cost of the min cost walk from s to it

$dist[x]$ = min cost ^{walk} from s to x found so far.

$prev$ = map associating each vertex to its predecessor

$prev[x]$ = the vertex just before x on the walk.

Time: $O(n \cdot m)$

Space: $O(n)$

relaxation operation: " if (x, y) edge s.t. $dist[y] > dist[x] + c(x, y)$
(better walk $\Rightarrow$ update)
 $dist[y] = dist[x] + c(x, y)$
 $prev[y] = x$

Algorithm:

for x in X do ; loop over all vertices

$dist[x] = \infty$; each vertex unreachable from s .

end for

$dist[s] = 0$; dist from s to itself

$changed = true$; suppose true

while $changed$ do ; keep looping as long as at least once $dist$ was updated previously

$changed = false$; suppose no change will happen

for (x, y) in E do ; loop over all edges $(x \rightarrow y)$

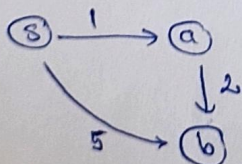
if $dist[y] > dist[x] + c(x, y)$ then
 $dist[y] = dist[x] + c(x, y)$
 $prev[y] = x$
 $changed = true$; updated $\Rightarrow$ true.] relaxation?

end if

end for

end while

Ex:



$dist[s] = 0$

$dist[a] = \infty$

$dist[b] = \infty$

$s \rightarrow a: 0 + 1 = 1 = dist[a]$

$a \rightarrow b: 1 + 2 = 3 = dist[b]$

$s \rightarrow b: \min(3, 5) = 3 = dist[b]$

Proof of correctness:

1) at each stage, $dist$ and $prev$ correspond to existing walks
 $dist[a] = 1$ comes from an actual path: $s \rightarrow a$.

2) algorithm finishes

? next iteration $\Rightarrow changed = false \Rightarrow$ stop.

? shortest paths never need more than $(n-1)$ edges.

3) when algorithm finishes, $dist[x] = d(s, x)$ for all x .

• after k iterations, we know the best paths using $\leq k$ edges.

$$dist[x] \leq w_k(s, x)$$

Time: $O(m \cdot m)$
Space: $O(m^2)$

Bellman Ford + dynamic programming ✓

$w_{k,x}$ = the cost of min. cost walk of length $\leq k$ from (s) to (x)
= ∞ if no such walk exists.

Recurrence relation:

$$\begin{cases} w_{0,s} = 0 \\ w_{0,x} = \infty, x \neq s \\ w_{k+1,x} = \min(\underbrace{w_{k,x}}, \min(\underbrace{w_{k,y}} + c(y,x))) \end{cases}, (\forall) k = \overline{0, m-2}$$

Final answer: $d(s,x) = w[m-1][x]$.

Algorithm:

for x in X do:

$w[0][x] = \infty$

end for

$w[0][s] = 0$

for $k = 0$ to $m-2$ do

for x in X do

$w[k+1][x] = w[k][x]$

$prev[k+1][x] = prev[k][x]$

end for

for (y,x) in E do

if $w[k][y] + c(y,x) < w[k+1][x]$ then

$w[k+1][x] = w[k][y] + c(y,x)$

$prev[k+1][x] = y$

endif

end for

end for

$dist[x] = w[m-1][x]$ for every vertex x .

A* algorithm (lowest cost walk)

A* = Dijkstra guided by a heuristic towards the target
↓
estimation $h(x)$.

- non-negative costs
- avoid computing paths that start from s but go in the opposite direction of t .
- estimation of the distance from s to t is needed. $\rightarrow$ input.

$$\rightarrow \text{strong condition: } \begin{cases} c(x,y) \geq h(x) - h(y) \\ h(t) = 0 \end{cases}$$

$$\rightarrow \text{weak condition: } h(x) \leq d(x,t)$$

- A* = Dijkstra but in the priority queue we have $\text{dist}[x] + h(x)$ instead of $\text{dist}[x]$.

Input: G = directed graph with costs

s, t = two vertices

$h: V \rightarrow \mathbb{R}$ = the estimation of the distance from s to t .

Output: dist = map associating to each vertex the cost of the min cost walk from s to it.

prev = map associating each vertex to its predecessor.

Algorithm: PriorityQueue q

Dictionary prev

Dictionary dist

$q.\text{enqueue}(s, h(s))$; put s in the priority queue with priority $h(s)$
 $\text{dist}[s] = 0$; distance from start to itself
 $\text{found} = \text{false}$

while not $q.\text{isEmpty}()$ and not found do

$x = q.\text{dequeue}()$

for y in $\text{Nout}(x)$ do

if y not in $\text{dist.keys}()$ or $\text{dist}[x] + \text{cost}(x,y) < \text{dist}[y]$ then
 $\text{dist}[y] = \text{dist}[x] + \text{cost}(x,y)$

$q.\text{enqueue}(y, \text{dist}[y] + h(y))$

$\text{prev}[y] = x$

end if

end for

if $x == t$ then

$\text{found} = \text{true}$

end if

end while

Proof of correctness:

$O(m^3)$ time.

✓ Floyd-Warshall (shortest cost walk between ALL vertices)

• negative cost edges allowed, but no negative cycles.

• $w_{k,i,j} = w_k(i,j)$ = shortest path from i to j using only vertices $\{1, \dots, k\}$

• DP over intermediate nodes.

• Floyd = Bellman-Ford for ALL pairs.

Base case:

$$\begin{cases} w_{l,x,x} = 0 \\ w_{l,x,y} = \text{cost}(x,y) \text{ if } \exists (x,y) \in E \\ w_{l,x,y} = \infty \text{ if } (x,y) \notin E \end{cases}$$

Recurrence:

$$w_{k+1}(i,j) = \min(w_k(i,j), w_k(i,k+1) + w_k(k+1,j))$$

$$\cancel{w_{k+l,x,y} = \min(w_{k,x,z} + w_{l,z,y})}$$

Algorithm: (vertices: $0, m-1$).

for $i=0$ to $m-1$ do

for $j=0$ to $m-1$ do

if $i=j$ then

$w[i,j] = 0$; dist from a node to itself = 0

else if (i,j) is edge in G then

$w[i,j] = \text{cost}(i,j)$; set its cost (dist)

$f[i,j] = j$; next node after i toward j is j .

else

$w[i,j] = \text{infinity}$

end if

end for

end for

for $k=0$ to $m-1$ do ; allow k as intermediate node

for $i=0$ to $m-1$ do

for $j=0$ to $m-1$ do

if $w[i,j] > w[i,k] + w[k,j]$ then ; checks if going through k is better

$w[i,j] = w[i,k] + w[k,j]$; update shortest distance

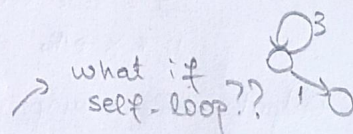
? $f[i,j] = f[i,k]$; next node after i toward j is the same as from i toward k .

end if

end for

end for

end for



Topological Sorting (predecessor counting) ✓

- DAG = directed acyclic graph
- find if there is any circular dependency (DAG or not)
- put the vertices in a list s.t. whenever $\exists$ edge (x, y) , then x comes before y in ^{the} list.
- take vertex with no predecessors → add it to the list → remove it from graph
- final: → no vertices left in the graph? → topologically sorted list.
→ we cannot get a vertex with no predecessors → cycle.
- (*) not actually remove, but keep a counter of its predecessors. :?

Input: G = directed graph

Output: Sorted = a list of vertices in topological sorting order
Null if G has cycles.

Algorithm:

```
sorted = emptylist ; list → answer
Queue  $q$  ; all vertices that currently have no predecessors left in-degree
Dictionary count ; count  $[x]$  = how many predecessors of  $x$  are still active
for  $x$  in  $X$  do ; all vertices
    count  $[x]$  = indeg  $(x)$  ; nr. of incoming edges into  $x$ 
    if count  $[x] = 0$  then
         $q.enqueue(x)$  ; put it first (no predecessors)
    endif
endfor
while not  $q.isEmpty()$  do ; continue as long as  $\exists$  at least one vertex with no predecessors
     $x = q.dequeue()$ 
    sorted.append( $x$ ) ;  $x$  placed in topological order
    for  $y$  in  $Nout(x)$  do ;  $x \rightarrow y$ 
        count  $[y] = count[y] - 1$  ; decrease nr of remaining predecessors
        if count  $[y] = 0$  then ; no remaining predecessors?
             $q.enqueue(y)$  ; processed later? next iteration?
        endif
    endfor
endwhile
if sorted.size() <  $X.size()$  then
    sorted = null ; we have a cycle
endif
```


(DFS) - Topo Sort ✓

• Murphy's law

do(x):

for y in Nim(x)

if y not yet done then → fast way to verify (set)

do(y)

endif

endfor

actually do x → list

same content as the list
quicker access by value

• do(x) invokes again do(x) before actually doing x?

Input: G = directed graph

Output: sorted = list of vertices in topological sorting order
NULL if G has cycles.

Subalgorithm TopoSortDFS(Graph G , ^{current vertex} x , ^{result list} List sorted, Set fullyProcessed, Set ^{currently explored} imProcess)

imProcess.add(x) ; currently explored

for y in Nim(x) ; inbound neighbours of x

if y in imProcess then

return false ; cycle

else if y not in fullyProcessed then

ok = TopoSortDFS(G , y , sorted, fullyProcessed, imProcess), explore y

if not ok then

return false ; cycle

endif

endif

endfor

imProcess.remove(x) ; remove x from current recursion stack

sorted.append(x) ; put x in result list

fullyProcessed.add(x) ; x is finished forever

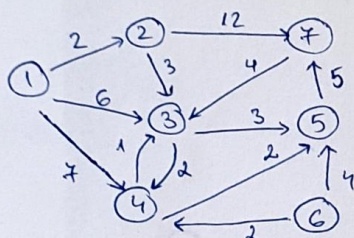
return true ; topo sort successful

• DAG, SCC, Preorder relations

• Scheduling problem

Written exam sample

? 1. Lowest cost path from all vertices to vertex x , reversed Dijkstra. (next)



- 1: $1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 7$ (cost 13)
- 2: $2 \rightarrow 3 \rightarrow 5 \rightarrow 7$ (cost 11)
- 3: $3 \rightarrow 5 \rightarrow 7$ (cost 8)
- 4: $4 \rightarrow 5 \rightarrow 7$ (cost 9)
- 5: $5 \rightarrow 7$ (cost 5)
- 6: $4 \rightarrow 5 \rightarrow 7$ (cost 9)
- 7: $7 \rightarrow 7$ (cost 0)

x	2	d	next
		1 2 3 4 5 6 7	1 2 3 4 5 6 7
7	7	$\infty \infty \infty \infty \infty \infty 0$	- - - - - - -
5	$\{2, 5\}$	$\infty 12 \infty \infty 5 \infty 0$	- 7 - - 7 - -
4	$\{1, 3, 4, 6\}$	$\infty 12 8 7 5 9 0$	- 7 5 5 7 5 -
3	$\{1, 3, 6\}$	$14 12 8 7 5 9 0$	4 7 5 5 7 5 -
6	$\{1, 2, 6\}$	$14 11 8 7 5 9 0$	4 3 5 5 7 5 -
2	$\{1, 2\}$	$14 11 8 7 5 9 0$	4 3 5 5 7 5 -
1	1	$13 11 8 7 5 9 0$	2 3 5 5 7 5 -
		$13 11 8 7 5 9 0$	2 3 5 5 7 5 -

2. DAG $\Rightarrow$ max length path, $O(n+m)$

$\Rightarrow$ topological sorting + d.p.

$x \mapsto d[x] = \text{longest path ending in } x$.

Recurrence: $\begin{cases} d[x] = \max(d[y] + 1), & \text{if } x \text{ has one predecessor.} \\ d[x] = 0, & \text{if } x \text{ has no predecessors} \end{cases}$

Input: $G = (X, E)$ a DAG.

Output: $p \rightarrow$ path of max length.

$x_0, \dots, x_{m-1} = \text{toposort}(G)$

for x in X do

$d[x] = 0$

 for $i = 0, n-1$ do

$x = x_i$

 for y in $N_{\text{out}}(x)$ do

 if $d[y] < d[x] + 1$ then

$d[y] = d[x] + 1$

$\text{maxLen} = -1$

 for x in X do

 if $d[x] > \text{maxLen}$ then

$\text{maxLen} = d[x]$

$\text{final} = x$

$x = \text{final}$

$p = []$

while $d[x] > 0$ do

$p.append(x)$

 for y in $N_{\text{in}}(x)$ do

 if $d[y] + 1 == d[x]$ then

$x = y$

 break

$p.append(x)$

$p.reverse()$

3. Min. spanning tree ; Prim's alg.

